

1. React Optimization

Why: Prevent unnecessary re-renders, improve app speed.

Techniques:

- **React.memo:** Prevents re-render if props haven't changed.

```
const MyComponent = React.memo(({ name }) => {  
  console.log("Rendered", name);  
  return <div>Hello {name}</div>;  
});
```

- **useMemo & useCallback:** Memoize expensive values and functions.

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);  
const memoizedFunction = useCallback(() => { doSomething(); },  
[dependency]);
```

- **Lazy loading / code splitting:** Load components only when needed.

```
const About = React.lazy(() => import('./About'));  
<Suspense fallback={<div>Loading...</div>}><About /></Suspense>
```

- **Avoid heavy calculations inside render.**
- **Keep component tree shallow.**

2. useEffect vs useLayoutEffect

Why: To run side-effects like fetching data, animations, DOM updates.

- **useEffect:** Runs after render.
- **useLayoutEffect:** Runs before browser paints.

```
useEffect(() => { console.log("after render"); }, []);
useLayoutEffect(() => { console.log("before paint"); }, []);
```

Use case: - useEffect → API calls, timers, logging. - useLayoutEffect → measure DOM, adjust layout before paint.

3. Functional vs Class Components

Functional Component: Simple, uses hooks.

```
function Greeting({ name }) {
  const [count, setCount] = useState(0);
  return <h1>Hello {name}, clicked {count} times</h1>;
}
```

Class Component: Old method, uses state and lifecycle.

```
class Greeting extends React.Component {
  state = { count: 0 };
  render() { return <h1>Hello {this.props.name}, clicked {this.state.count}
times</h1>; }
}
```

Why functional: Less code, easier with hooks.

4. React Rendering & Virtual DOM

How: 1. Virtual DOM = JS copy of real DOM. 2. State/props change → new Virtual DOM. 3. React diffs old vs new Virtual DOM. 4. Only changed parts update real DOM.

```
const [count, setCount] = useState(0);
<button onClick={() => setCount(count + 1)}>Click {count}</button>
```

Why: Efficient updates, avoids slow full DOM updates.

5. Closures in JS & React

Closure: Function remembers outer variables.

```
function outer() { let count = 0; return function inner() { count++;  
  console.log(count); } }  
const counter = outer(); counter(); counter();
```

React usage: Event handlers remember previous state.

```
const handleClick = () => setCount(count + 1);
```

6. Hoisting in JS

Definition: JS moves variable/function declarations to top.

```
console.log(a); // undefined  
var a = 5;
```

- Functions fully hoisted. - `let` / `const` not hoisted the same way.

7. Persisting Form Data

Problem: Refresh → data lost. Use localStorage.

```
const [name, setName] = useState(localStorage.getItem("name") || "");  
useEffect(() => { localStorage.setItem("name", name); }, [name]);  
<input value={name} onChange={e => setName(e.target.value)} />
```

Why: Data survives refresh.

8. Functional Calculator UI

Steps: 1. Convert Figma to HTML/CSS layout. 2. Use React state for input.

```
function Calculator() {
  const [input, setInput] = useState("");
  const handleClick = val => setInput(input + val);
  const calculate = () => setInput(eval(input));

  return (
    <div>
      <input value={input} readOnly />
      <button onClick={() => handleClick("1")}>1</button>
      <button onClick={() => handleClick("2")}>2</button>
      <button onClick={calculate}>=</button>
    </div>
  );
}
```

Why React: State updates automatically update UI.

9. React Keys

Why: Helps React identify items in a list, avoid unnecessary re-renders.

```
const fruits = ["Apple", "Banana"];
fruits.map((fruit, i) => <li key={i}>{fruit}</li>);
```

Without keys: React may re-render entire list → slow.

10. React Reconciliation

Definition: Comparing old vs new Virtual DOM and updating only changed parts. **Why:** Efficient updates, avoids full page re-render.

11. Virtual DOM & Performance

- Virtual DOM = lightweight JS copy.
 - Changes computed in memory → minimal real DOM update.
 - **Result:** Faster app even with large UI.
-

12. Code Splitting

Why: Reduce initial load, improve performance.

How: 1. React.lazy + Suspense

```
const About = React.lazy(() => import('./About'));  
<Suspense fallback={<div>Loading...</div>}><About /></Suspense>
```

2. Dynamic import

```
import('./MyComponent').then(module => { ... });
```

Result: Only needed code loads → faster.

Extra Tips for Beginners

- Keep state minimal.
 - Avoid heavy computations in render.
 - Use keys for lists.
 - Use hooks for state and side-effects.
 - Keep components simple and reusable.
-

End of Guide